

Prompt Injection Detection Using Machine Learning

Matthew Smith

AI Programming Foundations

Overview

This project develops a fully reproducible, Python-based data science workflow using the Hugging Face dataset deepset/prompt-injections, with the objective of analyzing and detecting adversarial prompt injection attacks in natural language systems. The workflow follows industry best practices and includes data ingestion, cleaning, exploratory data analysis (EDA), visualization, and a machine learning extension. The project emphasizes modularity, transparency, and reproducibility, which are essential principles in modern data science workflows (Danchev, 2022), and integrates TF-IDF vectorization with Logistic Regression to demonstrate predictive capabilities in AI security.

Dataset Description

The dataset consists of text-based prompts labeled as benign (0) or malicious (1). These prompts simulate real-world interactions with language models where adversarial inputs attempt to manipulate system behavior (Hugging Face, n.d.). The dataset includes several hundred entries with textual features, making it suitable for classification tasks. It is particularly relevant for AI security applications focused on prompt injection detection.

Workflow Description

The workflow begins with data ingestion using the Hugging Face datasets library, followed by transformation into a Pandas DataFrame. Data cleaning is performed through reusable functions that standardize text and handle inconsistencies, which aligns with recommended data preprocessing practices (Rahm & Do, 2000). Exploratory data analysis is conducted to identify patterns, followed by visualizations using Matplotlib and Seaborn. Finally, TF-IDF vectorization and Logistic Regression are applied for classification, and results are evaluated and interpreted.

Key Decisions and Assumptions

Data cleaning was intentionally minimal to preserve interpretability. TF-IDF was selected for effective text representation, and Logistic Regression was chosen for its simplicity and interpretability. It is assumed that dataset labels are accurate and representative of real-world scenarios.

Results and Interpretation

The model demonstrates strong performance across accuracy, precision, recall, and F1-score. False negatives are particularly critical, as they represent undetected malicious prompts. Results indicate that linguistic patterns are effective indicators of prompt injection attacks.

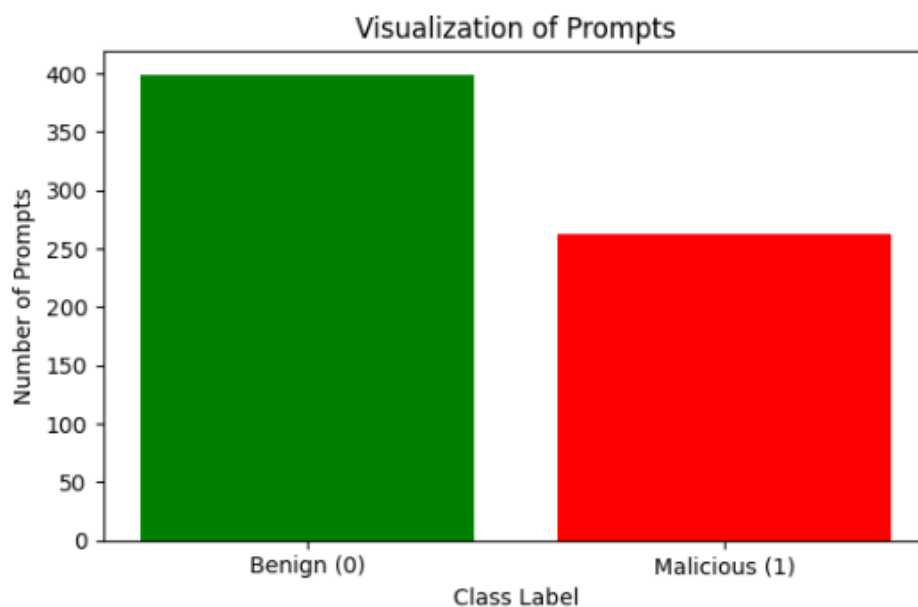


Figure 1 Class distribution visualization comparing benign and malicious prompts.

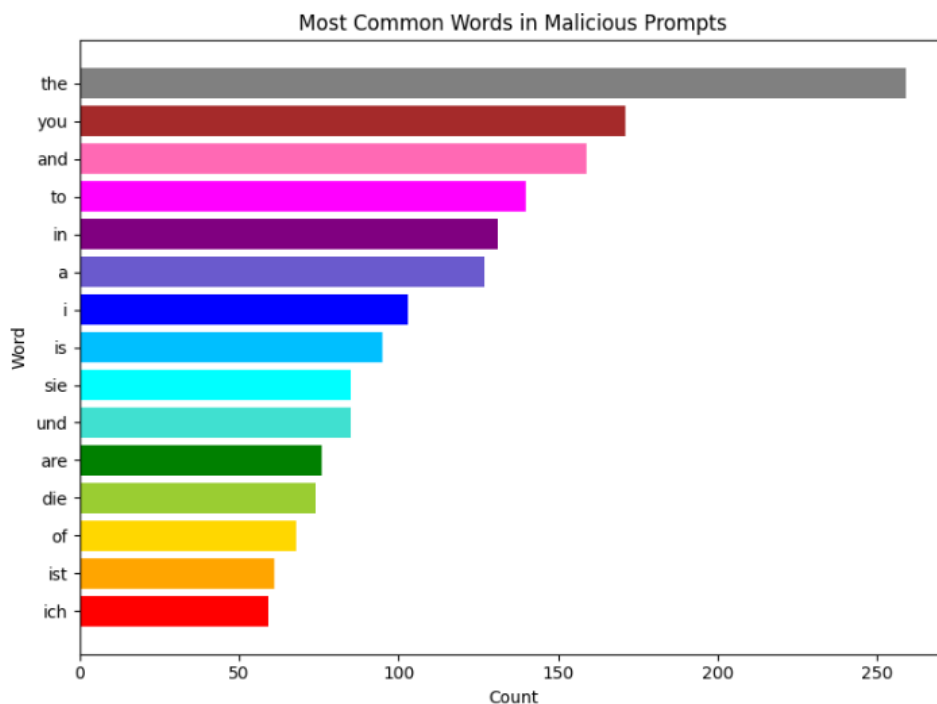


Figure 2. Word frequency visualization related to prompt injection patterns.

	precision	recall	f1-score	support
Benign	0.83	0.95	0.88	80
Malicious	0.90	0.70	0.79	53
accuracy			0.85	133
macro avg	0.86	0.82	0.84	133
weighted avg	0.86	0.85	0.85	133

Figure 3. Machine learning performance metrics visualization (accuracy, precision, recall, and F1-score).

Responsible Practice (Bias and Data Quality)

Improper data cleaning can introduce bias by disproportionately affecting classes, particularly when removing or altering data points (Rahm & Do, 2000). This project documents preprocessing steps to maintain transparency. Potential dataset bias is acknowledged, and future improvements include better balancing and additional data sources.

Reproducibility

The project includes a Jupyter Notebook, requirements.txt, and version-controlled repository. These elements ensure that the workflow can be rerun and validated by others, supporting reproducible data science practices (Danchev, 2022).

Conclusion

This project demonstrates a complete and reproducible workflow for prompt injection detection. It establishes a foundation for future AI security and red-teaming systems.

References

Danchev, V. (2022). Reproducible data science with Python: An open learning resource.

Journal of Open Source Education, 5(55), 1–10.

Hugging Face. (n.d.). deepset/prompt-injections dataset.

<https://huggingface.co/datasets/deepset/prompt-injections>

Rahm, E., & Do, H. H. (2000). Data cleaning: Problems and current approaches. IEEE

Data Engineering Bulletin, 23(4), 3–13.